

# BDBP-309 Deep Learning Laboratory - Notes & Exercises

Shyam Rajagopalan,  
IBAB, Bengaluru

20 July 2026

In God we trust, all others bring data.

-William Edward Deming (1900-1993)

## Lab 1 - Activation Functions

### Learning Goals

In this lab, we will look at different functions used for introducing non-linearity in neural networks.

By the end of this lab, you should be able to

1. Understand different types of activation functions that are useful to introduce non-linearity into a neural network.
2. the pros and cons of different types of activation functions
3. How exactly it is implemented in standard python libraries

### Exercises

1. Implement the following functions in Python from scratch. Do not use any library functions. You are allowed to use numpy and matplotlib. Generate 100 equally spaced values between -10 and 10. Call this list as `z`. Implement the following functions and its derivative. Use class notes to find the expression for these functions. Use `z` as input and plot both the function outputs and its derivative outputs. Upload your code into Github and share it with me.
  - a. Sigmoid
  - b. Tanh
  - c. ReLU (Rectified Linear Unit)
  - d. Leaky ReLU
  - e. Softmax (no need for visualization)

2. Write down the observations from the plot for all the above functions in the code.
  - a. What are the min and max values for the functions?
  - b. Is the output of the function zero-centred?
  - c. What happens to the gradient when the input values are too small or too big?
  - d. What is the relationship between sigmoid and tanh?

## Lab 2 - Forward Pass

### Learning Goals

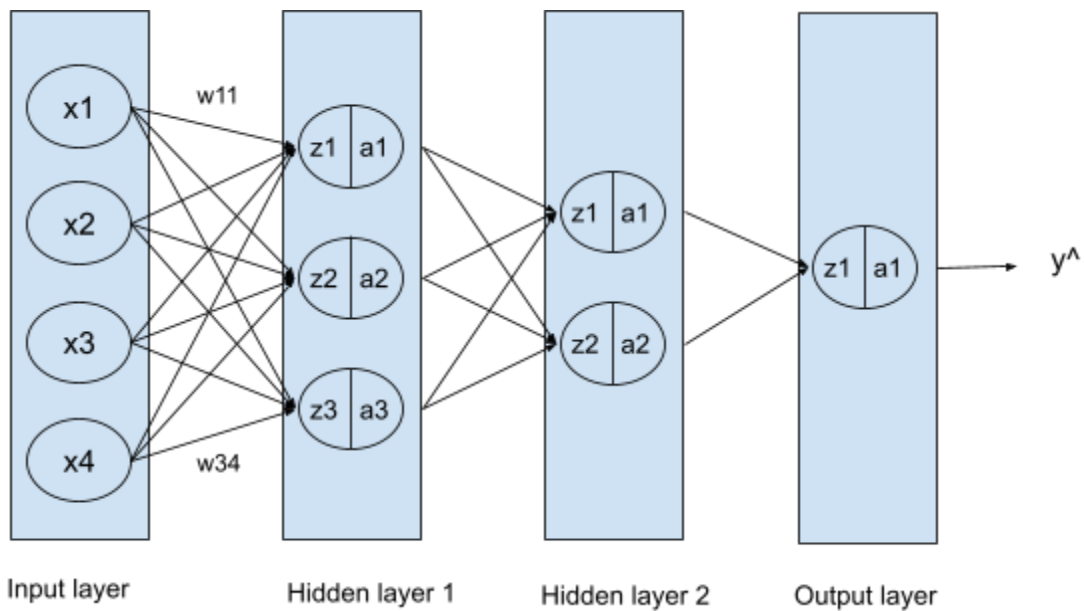
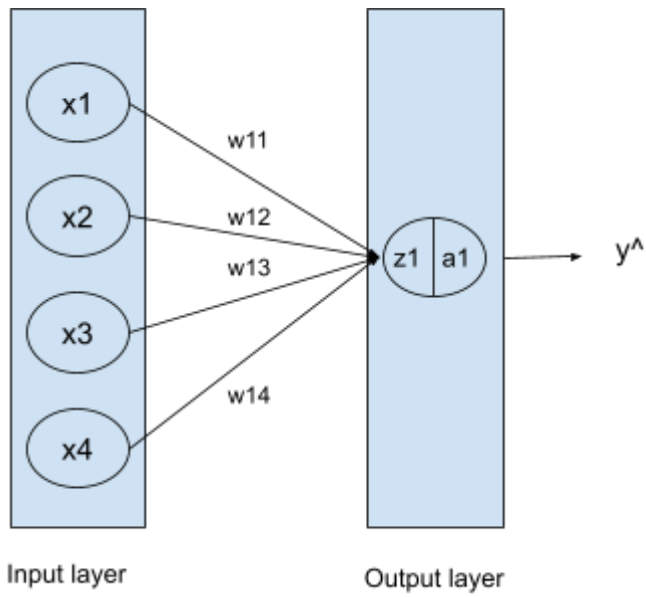
In this lab, we will look at the mechanism of forward pass in a neural network.

By the end of this lab, you should be able to

1. Appreciate how the input data flows into the network, how they get transformed and finally leads to computing the target value or the output.
2. Understand various parameters and hyperparameters used in the neural network, their dimensions and how they interact with each other.
3. Become familiar with writing loops and vector operations for computing neuron activations.
4. How activation functions help in introducing non-linear operations and overall how a single neuron works.

### Exercises

1. Consider the following two networks.  $W$  is a matrix,  $x$  is a vector,  $z$  is a vector, and  $a$  is a vector.  $y^{\wedge}$  is a scalar and a final prediction. Initialize  $x$ ,  $w$  randomly,  $z$  is a dot product of  $x$  and  $w$ ,  $a$  is  $\text{ReLU}(z)$ . Initialize  $X$  and  $W$  randomly. Every neuron has a bias term.



5. Implement forward pass for the above two networks. Print activation values for each neuron at each layer. Print the loss value ( $\hat{y}$ ).
3. Implement the forward pass using vectorized operations, i.e.  $W$  should be a matrix,  $x, z$  and  $a$  are vectors. The implementation should not contain any loops.

# Lab 3 - Backward pass - Backpropagation

## Learning Goals

In this lab, we will look at the mechanism of backward pass in a neural network. This is a critical component of training a neural network.

By the end of this lab, you should be able to

1. Learn the notion of computational graph and how to compute gradients for different parts of the computational graph.
2. Understand how the gradient flows backwards in a neural network across multiple layers
3. Familiarize yourself with the stochastic gradient descent algorithm for parameter updates in a neural network.

## Exercises

1. Implement backward pass for the above two networks. Print the gradient values for each neuron in each layer.

# Lab 4 - Full Neural Network Implementation from Scratch - No deep learning libraries

## Learning Goals

In this lab, you will learn to implement end-to-end neural network ground-up without using any deep learning libraries.

By the end of this lab, you should be able to

1. Build a working neural network model integrating all components (layers, activation, forward and backward pass, and loss functions) of the architecture.
2. Learn to code all parts of the neural network from scratch. This will help you to deeply understand the learning procedure.
3. Understand the training process, batch processing, epochs and testing procedure.

## Exercises

1. Implement a 1-layer (input - output layer) neural network from scratch for the following dataset. This includes implementing forward and backward passes from scratch. Print the training loss and plot it over 1000 iterations.

| x1 | x2 | x3 | y |
|----|----|----|---|
| 0  | 0  | 1  | 0 |
| 1  | 1  | 1  | 1 |
| 1  | 0  | 1  | 1 |
| 0  | 1  | 1  | 0 |

4. Visualize at the weights and gradients of the layers -  
<https://machinelearningmastery.com/visualizing-the-vanishing-gradient-problem/>
5. <https://wandb.ai/ayush-thakur/debug-neural-nets/reports/Visualizing-and-Debugging-Neural-Networks-with-PyTorch-and-Weights-Biases--Vmlldzo2OTUzNA>
6. <https://karpathy.github.io/2019/04/25/recipe/>

## Lab 5 - XOR implementation from Scratch - No deep learning libraries

### Learning Goals

In this lab, you will understand the idea of implementing functions using a neural network.

By the end of this lab, you should be able to

1. Build a working neural network model for implementing XOR operation.
2. Familiarize yourself with the training process and how a neural network can approximate learning functions.

### Exercises

1. Implement a 2-layer (input layer, hidden layer and output layer) neural network from scratch for the XOR operation. This includes implementing forward and backward passes from scratch. The truth table for XOR is given by

| x1 | x2 | y (x1 XOR x2) |
|----|----|---------------|
| 0  | 0  | 0             |
| 0  | 1  | 1             |

|   |   |   |
|---|---|---|
| 1 | 0 | 1 |
| 1 | 1 | 0 |

## Lab 6 - Getting started with PyTorch

### Learning Goals

In this lab, you will familiarize yourself with one of the most popular deep learning libraries, PyTorch.

By the end of this lab, you should be able to

1. install PyTorch and familiarize yourself with various data structures in PyTorch

### Exercises

1. Install PyTorch (<https://pytorch.org/>) with CPU option selected, torch, torchvision
2. Learn about Tensors, Datasets and Dataloaders, Transforms, BuildModel, Autograd, Optimization and Save & load model - Try the code snippets for each of the above operations from here - <https://pytorch.org/tutorials/beginner/basics/intro.html>
3. Loading custom dataset - [https://pytorch.org/tutorials/beginner/data\\_loading\\_tutorial.html](https://pytorch.org/tutorials/beginner/data_loading_tutorial.html)

## Lab 7 - Feedforward Network using CPU and GPU

### Learning Goals

In this lab, you will begin to learn deep network model development using PyTorch. From this point, you will start using PyTorch for all exercises (unless specified otherwise explicitly) to build deep networks.

By the end of this lab, you should be able to

1. Build a simple deep neural network (called deep network henceforth) classifier using PyTorch.
2. Train a deep network model, learn when to stop training, how to use batches and how to tune hyperparameters.

### Exercises

1. Build an end-to-end working deep learning model using PyTorch library - [https://pytorch.org/tutorials/beginner/basics/quickstart\\_tutorial.html](https://pytorch.org/tutorials/beginner/basics/quickstart_tutorial.html)

## Lab 8 - Dropouts, BatchNorm and Optimizations

### Learning Goals

In this lab, you will learn key tools needed for successfully training a deep network.

By the end of this lab, you should be able to

1. Handle overfitting issues using regularization techniques
2. Understand batch and layer normalization
3. familiarize yourself with various optimization techniques.

### Exercises

1. Implement batch normalization from scratch. and layer normalization for training deep networks.
2. Implement dropout to regularize neural networks from scratch.
3. Implement various update rules used to optimize the neural network. You can use PyTorch for the following implementations.
  - a. SGD, Momentum, AdaGrad, etc.

## Lab 9 - Vanishing & Exploding Gradient Problem

### Learning Goals

In this lab, problems associated with training a very deep network will be studied.

By the end of this lab, you should be able to

1. Understand the idea of vanishing and exploding gradients
2. Apply mechanisms to address vanishing and exploding gradients

### Exercises

1. Write a program to simulate vanishing & exploding gradient problems.  
<https://machinelearningmastery.com/visualizing-the-vanishing-gradient-problem/>

2. Plot the gradient values to demonstrate the issues with training a very deep network.

## Lab 10 - Deep learning model for target gene prediction from a set of landmark genes

### Learning Goals

In this lab, you will apply your learnings on feed forward deep networks to a biological problem of predicting target genes.

By the end of this lab, you should be able to

1. Successfully implement a deep learning model to solve a biological problem.
3. Process biological datasets to develop predictive models.

### Exercises

1. Develop a model to predict target genes from a set of landmark genes - <https://academic.oup.com/bioinformatics/article/32/12/1832/1743989> <https://sites.google.com/view/cs502project/deep-learning-for-biology-a-tutorial/biological-data-in-deep-learning> .
2. Refer to the appropriate section and the paper cited in this article. Look at the paper and download the dataset used in the paper.
3. Implement a deep network model and see how your result compares with the one presented in the paper.
4. Explore different ways of encoding input data.
5. Tune hyperparameters for improved performance.

## Lab 11 - Convolutional Neural Networks (CNN) Implementation from scratch

### Learning Goals

In this lab, you will get introduced to components of a CNN.

By the end of this lab, you should be able to

1. Learn the idea of convolution operation along with various kernels and sizes.
2. familiarize yourself with the maxpool operation and how it helps in reducing the dimensionality.

3. Understand how the input sizes gets transformed after convolution and max pooling operations.

## Exercises

1. Implement convolution operations from scratch. Assume a 3x3 kernel and apply it on an input image of 32x32.
2. Implement maxpool operation from scratch.

# Lab 12 - CNN Applications

## Learning Goals

Having learnt the basic components of a CNN, in this lab, you will implement an end-end deep network using CNN.

By the end of this lab, you should be able to

1. Develop a deep learning model using CNN for analysing image data.
2. Deeply understand how the kernel and convolution operations allow us to automatically learn features.
3. Thoroughly understand the dimension of parameters, input and output.

## Exercises

1. Download MNIST dataset and implement a MNIST classifier using CNN PyTorch library.
2. Implement CNN using PyTorch for image classification using cifar10 dataset - [https://pytorch.org/tutorials/beginner/blitz/cifar10\\_tutorial.html](https://pytorch.org/tutorials/beginner/blitz/cifar10_tutorial.html) . Plot train error vs increasing number of layers. After some point, the training error increases with the number of layers.

# Lab 13 - Predicting Gene Accessibility using CNNs

## Learning Goals

In this lab, you will apply CNN for solving gene accessibility problems.

By the end of this lab, you should be able to

1. Develop a deep learning model using CNN to process genomic data and develop a prediction model for gene accessibility.
2. Learn to transform the input sequence data to suitable representations.

## Exercises

1. Develop a prediction model for gene accessibility prediction using CNNs - <https://sites.google.com/view/cs502project/deep-learning-for-biology-a-tutorial/a-deep-dive-into-a-convolutional-neural-network-example?pli=1>
2. Explore different sequence representations and write a technote of your findings.

# Lab 14 - Recurrent Neural Network (RNN)

## Learning Goals

In this lab, you will learn the basic architecture of RNN.

By the end of this lab, you should be able to

1. Form intuitions about processing sequence or time-series data
2. Understand how RNN works by unwrapping the sequence information

## Exercises

1. Implement vanilla RNN from scratch - <https://medium.com/@thisislong/building-a-recurrent-neural-network-from-scratch-ba9b27a42856>

# Lab 15 - RNN Applications

## Learning Goals

In this lab, you will learn commonly solved problems of Image captioning using RNN and LSTM

By the end of this lab, you should be able to

1. Build a RNN for image caption problem
2. Build Long Short-Term Memory (LSTM) model for image caption problem.

## Exercises

1. Implement vanilla RNN using PyTorch for image captioning. You can use the PyTorch library.
2. Implement LSTM using PyTorch for image captioning

## Lab 16 - Sequence to Sequence Learning using RNN

### Learning Goals

In this lab, you will learn the principles behind sequence to sequence learning problems.

By the end of this lab, you should be able to

1. Understand the encoder-decoder architecture commonly used in sequence to sequence learning problems.
2. Compare LSTM and GRU's performances.

## Exercises

1. Implement sequence to sequence learning using LSTM and GRU. You can use the the English to Hindi & Hindi to English corpus dataset - <https://www.kaggle.com/code/aiswaryaramachandran/english-to-hindi-neural-machine-translation>
2. Study the performance between LSTM and GRU.
3. Demonstrate how both LSTM and GRU solves the vanishing gradient problem.

## Lab 17 - Predicting protein function from protein sequence using RNN

### Learning Goals

In this lab, you will develop a deep network model to predict protein function from sequences

By the end of this lab, you should be able to

1. Process protein sequence data and encode them into an embedding vector

2. Develop a prediction model of protein function.

## Exercises

1. Develop a deep learning model for predicting protein function using protein sequence as an input - <https://arxiv.org/abs/1701.08318> and <https://sites.google.com/view/cs502project/deep-learning-for-biology-a-tutorial/biological-data-in-deep-learning> Also, please check <https://www.kaggle.com/competitions/cafa-6-protein-function-prediction/overview> (Thanks to Surath Mounika for passing this reference)

# Lab 18 - Embeddings

## Learning Goals

In this lab, you will develop a deep network model to predict protein function from sequences

By the end of this lab, you should be able to

1. Understand the idea of embedding and how they are used to form efficient representations of different data modalities.

## Exercises

2. Explore different representation learning algorithms for text, image and biological datasets.

# Lab 19 - Generative Adversarial Networks (GAN & Stable Diffusion)

## Learning Goals

In this lab, you will get introduced to generative models, such as GAN and Stable Diffusion.

By the end of this lab, you should be able to

1. Understand the differences between discriminatory and generative models
2. Learn about GANs and Stable Diffusion methodologies.

## Exercises

1. Implement GAN using PyTorch for image caption
2. Develop a stable diffusion based method for image caption generation.

## Lab 19 - Transformers (Additional Material - Not in Syllabus)

1. Implement Transformers using PyTorch for image caption

## Lab 20 - Self-supervised Learning (Additional Material - Not in Syllabus)

1. Implement Self-supervised Learning using PyTorch for image classification

## Lab 21 - Foundation Models (Additional Material - Not in Syllabus)

1. Finetune medical foundation models on a custom dataset